Computer Security Exam
Professors M. Carminati & S. Zanero

Milan, 10/09/2020

Last (family) Name _____

First (given) Name _____

Matricola or Codice Persona _____

Have you done any challenges/homework, even partially? [ ] Yes [ ] No
Professor: [ ] Carminati [ ] Zanero
Course: [ ] Milan [ ] Como

Instructions:
- Duration: 2 hours.
- The exam is "open book". The students will be allowed to consult only the course material and notes. Be aware that notes and course materials will be accepted only on paper form. No extra devices (e.g., phones, iPad) are allowed.
- You are not allowed to communicate with other students in any communication form. You will be expelled if, at any time, you do not follow this rule and will not be allowed to take the next session.
- Schemes are good, short answers are recommended.
- Always motivate your answer.
- If your final grade is below 120 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 (140 points)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <stdint.h>
5
6 const uint32_t bird = 0x41424344;
7
8 typedef struct {
9     unsigned short backdoor;
10     int auth;
11     int *fd;
12     char name[8];
13     char pass[12];
14     char realpass[12];
15     uint32_t bird;
16 } data_t;
17
18 int verify_user(data_t* data) {
19     char message[64];
20     // <-- HERE
21     data->fd = fopen("pass.txt", "r");
22     fscanf(data->fd, "%12s", data->realpass);
23     fclose(data->fd);
24
25     scanf ("%64s", data->pass);
26     if (data->bird != bird)
27         abort();
28     if(strncmp(data->pass, data->realpass, 12) == 0 || data->backdoor == 0) {
29         printf("PASSWORD OK\n");
30         return 1;
31    } else {
32         snprintf(message, 64, "Wrong password :%s\n", data->pass);
33         printf(message);
34     }
35     return 0;
36 }
37
38 void auth(int token) {
39     data_t data;
40
41     data.auth = 0;
42    data.bird = bird;
43     data.backdoor = token;
44
45     scanf ("%20s", data.name);
46     printf("Welcome! %s\n", data.name);
47
48     data.auth = verify_user(&data);
49     if (data.auth == 1) {
50         printf("SUCCESS!\n");
51         return;
52     }
53     printf("FAIL!!\n");
54     abort();
55 }
56
57 void main(int argc, char** argv) {
58     int token;
59     token = atoi(argv[1]);
60     if (token == 0)
61         abort();
62     auth(token);
63 }
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>

const uint32_t bird = 0x41424344;

typedef struct {
    unsigned short backdoor;
    int auth;
    int *fd;
    char name[8];
    char pass[12];
    char realpass[12];
    uint32_t bird;
} data_t;

int verify_user(data_t* data) {
    char message[64];
    // <-- HERE
    data->fd = fopen("pass.txt", "r");
    fscanf(data->fd, "%12s", data->realpass);
    fclose(data->fd);

    scanf ("%64s", data->pass);
    if (data->bird != bird)
        abort();
    if(strncmp(data->pass, data->realpass, 12) == 0 || data->backdoor == 0) {
        printf("PASSWORD OK\n");
        return 1;
    } else {
        snprintf(message, 64, "Wrong password :%s\n", data->pass);
        printf(message);
    }
    return 0;
}

void auth(int token) {
    data_t data;

    data.auth = 0;
    data.bird = bird;
    data.backdoor = token;

    scanf ("%20s", data.name);
    printf("Welcome! %s\n", data.name);

    data.auth = verify_user(&data);
    if (data.auth == 1) {
        printf("SUCCESS!\n");
        return;
    }
    printf("FAIL!!\n");
    abort();
}

void main(int argc, char** argv) {
    int token;
    token = atoi(argv[1]);
    if (token == 0)
        abort();
    auth(token);
}
```

[5 points] 08. The program may be affected by a typical buffer overflow vulnerability. Specify the vulnerable line/s of code. If more than one line is affected, use the following format LINE1, LINE2,..,LINEN (e.g., 00,01,03). Write "no vulnerabilities" in case a class of vulnerabilities is not present.

25, 45

[5 points] 09. Motivate the previous answer.

...

[5 points] 10. The program may be affected by a typical format string vulnerability. Specify the vulnerable line/s of code. If more than one line is affected, use the following format LINE1, LINE2,..,LINEN (e.g., 00,01,03). Write "no vulnerabilities" in case a class of vulnerabilities is not present.

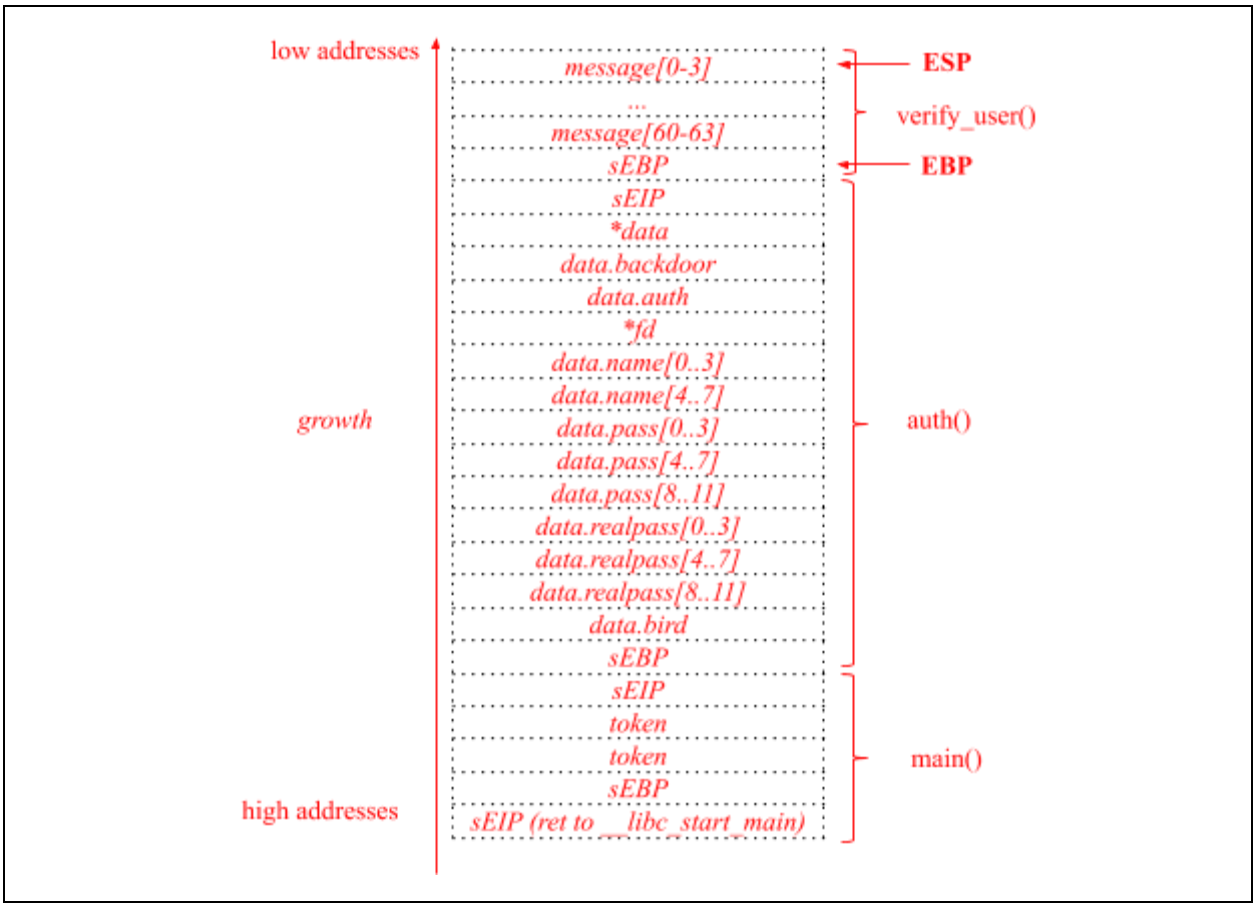
33

[5 points] 11. Motivate the previous answer.

...

[10 points] ***12.*** *From now on, assume the program is compiled for the* ***x86 architecture*** *(32 bit) and for an environment that adopts the usual* **`cdecl`** *calling convention. Furthermore, assume that* ***no compiler-level or OS-level mitigations*** *against the exploitation of memory corruption errors is present (unless specified otherwise).* Draw the stack layout after the program has executed the instruction at line 20, showing:
- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

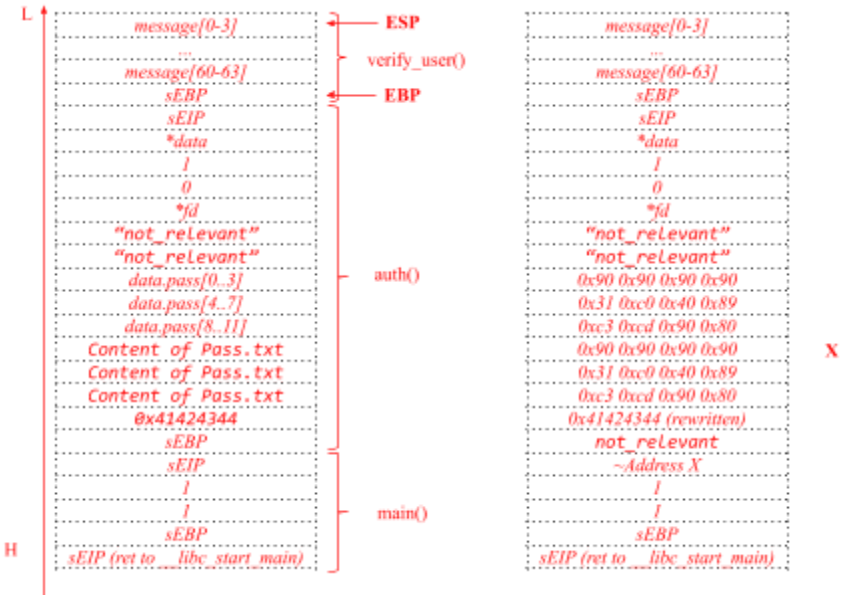
Show also the content of the caller frame.



[20 points] 13. Focus only on the **stack-based buffer overflow(s)** you found at Point 1. Write an exploit for this vulnerability that bypasses the verify_user() function. Your exploit **must execute the following shellcode**, composed of 8 bytes, which bypasses the authentication: **`0x31 0xc0 0x40 0x89 0xc3 0xcd 0x90 0x80`**.

Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation. Ensure you describe **all the steps and assumptions** required for a successful exploitation of the vulnerability. Include also any assumption on how you must call the program (e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables, if any).

Argv[1] any number != 0
data.name -> “not relevant”
Data.pass -> exploit



[10 points] 14. Assume that the program is compiled only with the following custom implementation of the compiler-level mitigation known as "XOR Stack Canary" (the address space layout is not randomized, and the stack is executable):

- **Function prologue**: at the very beginning of each function, the value of the global variable "*bird*" is xored with the values of the just pushed sEIP and the sEBP. The resulting value is then pushed on the stack in the usual stack canary position (i.e., between the sEBP and the local variables).
- **Function epilogue**: Just before returning from the function, the value stored on the stack is compared with the one which is computed following the same procedure.

This way, if an attacker tries to change any part of these control elements of the stack, the value of the canary will change and, during the function epilogue, the execution will fail.
Unfortunately, the mitigation is implemented wrongly. Please explain why this specific implementation is not effective, how an attacker can exploit it, and describe how you would resolve the issue(s).

It must be random...

[10 points] 15. Now focus on the format string vulnerability only. Write an exploit for this vulnerability that executes the previous shellcode, assuming that you have already allocated the shellcode in memory. Assume that:

- The address of the return address (*saved EIP*) of the function **`auth`** is: **`0xffffb5d7`** (i.e., where to write)
- The address of the first instruction of the **`shellcode`** is at: **`0xf4d0f4d0`** (i.e., what to write)
- The displacement on the stack of the vulnerable function's argument is: **4**

Knowing that *dec(0xf4d0) = 62672*, write the exploit clearly, detailing all the components of the format string and all the steps that lead to a successful exploitation.

To exploit it, we need to write 0xf4d0f4d0 to 0xffffb5d7.
As 0xf4d0 == 0xf4d0 it is not relevant to swap or not.
The general format string structure is:
<where to write><where to write + 2>%<N1>c%<pos>$hn%<N2>c%<pos>$hn
- Where to write = \xd7\xb5\xff\xff
- Where to write + 2 = \xd9\xb5\xff\xff
- we have already written (16 characters) + 8 characters
- N1 -> 62672 - 16 - 8 = 62648
- N2 -> 62672 - 62672= 0
Also, the displacement on the stack of the printf's argument (i.e., the buffer message) is 2, the displacement of "where to write" is 4 + 4 = 8 and the displacement of "where to write + 1" is 9.

Complete exploit: **\xd7\xb5\xff\xff\xd9\xb5\xff\xff%62648c$8hn$9hn**

[5 points] 16. Consider the correct implementation of "Stack Canary" mitigation technique. It is effective to prevent **your exploits** at point 13.?

yes

[5 points] 17. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

See slides + can be bypassed using the format string exploit that directly overwrites the sEIP.

[5 points] 18. Consider the "Stack Canary" mitigation technique. It is effective to prevent **your exploits** at point 15.?

No,

[5 points] 19. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

See slides + the format string exploit directly overwrites the sEIP

[5 points] 20. Consider the "Address-space layout randomization (ASLR)" mitigation technique. It is effective to prevent **your exploits** at point 13.?

Yes

[5 points] 21. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

the exploit assumes that the address of the buffer is known, at least with a certain degree of precision (assuming that the entropy of the randomisation is enough)
If ASLR randomizes only the libraries and the stack\heap, using ROP with gadgets in the text section. If the binary is PIE, we can bypass ASLR only if there's another vulnerability which gives a leak of some stack or libc address. In this case, we could use the format string vuln as a leak (see answer to question 23).

[5 points] 22. Consider the "Address-space layout randomization (ASLR)" mitigation technique. It is effective to prevent **your exploits** at point 15.?

Yes

[5 points] 23. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

same reason as the buffer overflow

[5 points] 24. Consider the "Non-executable stack (NX)" mitigation technique. It is effective to prevent **your exploits** at point 13.?

Yes

[5 points] 25. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

the exploit needs to execute shellcode from the stack which wouldn't be executable. + Return to libc or ROP.

[5 points] 26. Consider the "Non-executable stack (NX)" mitigation technique. It is effective to prevent **your exploits** at point 15.?

Yes

[5 points] 27. Consider the previous answer. If the mitigation technique is effective, explain why and describe how you would modify the exploit (or use a different exploitation technique) to bypass the mitigation. If it is not effective, please explain why.

The exploit involves the execution of any shellcode from writable memory areas. ..ROP or ret2libc

[10 points] 28. Assume now that all the mitigation techniques analyzed before are correctly enabled (i.e., Stack Canaries, "Non-executable stack (NX)", "Address-space layout randomization (ASLR)").
Is it still possible to bypass the authentication mechanism implemented? If yes, explain why and describe how you would modify the previous exploits (or exploit a different vulnerability) . If not, please explain why.

Logic Vulnerability (integer overflow?) -> Argv[1] = 65536
But also realpass == pass

Exercise 2 (90 points)

"Have a Sitar" is an e-commerce website where users can buy virtual money and either spend it on objects or gift it to friends. Objects will be immediately sent to your house as you buy them! It also has a cool page that shows the best sellers for each month. The relevant code of the web-app is the following:

```
#Snippet 1: Given a month, it displays the best seller - https://have.a.sitar.com/best_seller
def best_seller(request):
    user = check_session(request.cookies['session'])
    if user is None or request.method != "GET":
        abort(403)
    best_seller = query("select best_seller from Stats where month='" + request.month + "';")
    if best_seller is None:
        abort(404)
    page = "<h1> The best seller of the month you've asked for was " +  htmlentities(best_seller) + " ! </h1>"
    return render(page)

#Snippet 2: It allows a user to buy an object - https://have.a.sitar.com/buy
def buy_object(request):
    user = check_session(request.cookies['session'])
    if user is None or request.method != "GET":
        abort(403)

    beginSQLtransaction()
    q1 = prepared_stmt("select credit from Users where username = :user")
    user_money = query(q1, user)
    q2 = prepared_stmt("select price from Store where object = :obj")
    price = query(q2, request.obj)
    credit_left = user_money - price
    if credit_left < 0:
        endSQLtransaction()
        abort(403)
    q3 = prepared_stmt("update Users set credit=:credit_left where username = :user")
    query(q3, credit_left, user)
    manage_shipment(user, request.obj) # Other SQL queries for shipments
    endSQLtransaction()

    page = "<h1> Hey, good choice! </h1>"
    return render(page)

#Snippet 3: It allows a user gift money to another user - https://have.a.sitar.com/gift
def send_gift(request):
    user = check_session(request.cookies['session'])
    if user is None or request.method != "GET":
        abort(403)
    #Check if the user has enough money
    q1 = prepared_stmt("select credit from Users where username = :user")
    user_money = query(q1, user)
    if user_money < request.amount:
        abort(403)
    money_left = user_money - request.amount
    #Make the donation
    q2 = prepared_stmt("update Users set credit=:money_left where username = :user ")
    q3 = prepared_stmt("update Users set credit=credit+:amount where username = :to")
    query(q2, money_left, user)
    query(q3, request.to, request.amount)
    page = "<h1> Money sent! </h1>"
    return render(page)
```

1 Snippet 1: Given a month, it displays the best seller - https://have.a.sitar.com/best_seller

```
2 def best_seller(request):
3       user = check_session(request.cookies['session'])
4       if user is None or request.method != "GET":
5           abort(403)
6       best_seller = query("select best_seller from Stats where month='" + request.month + "';")
7       if best_seller is None:
8               abort(404)
9       page = "<h1> The best seller of the month you've asked for was " +
htmlentities(best_seller) + " ! </h1>"
10      return render(page)
11
```

12 Snippet 2: It allows a user to buy an object - https://have.a.sitar.com/buy

```
13 def buy_object(request):
14     user = check_session(request.cookies['session'])
15     if user is None or request.method != "GET":
16          abort(403)
17
18      beginSQLtransaction()
19      q1 = prepared_stmt("select credit from Users where username = :user")
20      user_money = query(q1, user)
21      q2 = prepared_stmt("select price from Store where object = :obj")
22      price = query(q2, request.obj)
23      credit_left = user_money - price
24      if credit_left < 0:
25              endSQLtransaction()
26              abort(403)
27      q3 = prepared_stmt("update Users set credit=:credit_left where username = :user")
28      query(q3, credit_left, user)
29      manage_shipment(user, request.obj) # Other SQL queries for shipments
30      endSQLtransaction()
31
32      page = "<h1> Hey, good choice! </h1>"
33      return render(page)
34
```

35 Snippet 3: It allows a user gift money to another user - https://have.a.sitar.com/gift

```
36 def send_gift(request):
37     user = check_session(request.cookies['session'])
38     if user is None or request.method != "GET":
39          abort(403)
40    #Check if the user has enough money
41    q1 = prepared_stmt("select credit from Users where username = :user")
42    user_money = query(q1, user)
43    if user_money < request.amount:
44      abort(403)
45    money_left = user_money - request.amount
46    #Make the donation
47    q2 = prepared_stmt("update Users set credit=:money_left where username = :user ")
48    q3 = prepared_stmt("update Users set credit=credit+:amount where username = :to")
49    query(q2, money_left, user)
50    query(q3, request.to, request.amount)
51    page = "<h1> Money sent! </h1>"
52    return render(page)
```

Clearly, the website uses a relational database as a back-end. The database schema can be inferred from the following sample tables:

Users

username	password	credit
Emily	Misunderstands	50
Pink	Worms	0
Lucifer	Sam	5
Matilda	Mother	5
...	...	

Stats

best_seller	month
Bowl	June
Bike	May
Lap-steel	April
Sitar	March
...	...

Store

object	price
Bike	20
Overdrive	100
Cigar	2
Poem Book	10
...	...

Also, assume the following:
- the function check_session() securely manages the users' sessions
- the function htmlentities() correctly escapes all the html special characters.
- it is not possible to register a user twice (i.e., usernames are unique in the system) and all usernames contain at least one character.
- each user starts with a "free" credit of £5.
- the library used to access the DBMS does not support stacked queries, i.e., the DBMS does not execute multiple queries if asked on the same query and separated by a semicolon. For instance, the query "select x from y where a=b; update k set [....] " would trigger an error and not be executed.

Please read the following questions and then answer one by one.

[5 points] 29. Focus on SQL injection. Is there a vulnerability belonging to this class in the code?

Yes

[5 points] 30. Motivate the previous answer: If the vulnerability is present, explain: -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability *while preserving the intended functionalities of the page*. If the vulnerability is not present, explain why.

a) The query in Snippet 1 b),c),d) see slides

[5 points] 31. Focus on Cross-site scripting (XSS). Is there a vulnerability belonging to this class in the code?

No

[5 points] 32. Motivate the previous answer: If the vulnerability is present, explain: -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability *while preserving the intended functionalities of the page*. If the vulnerability is not present, explain why. (please also specify which subclasses of XSS are present, e.g., stored\reflected\DOM-based)

All the page contain static strings, the only user-input value is html-escaped.

[5 points] 33. Focus on Cross site request forgery (CSRF). Is there a vulnerability belonging to this class in the code?

yes

[5 points] 34. Motivate the previous answer: If the vulnerability is present, explain: -- (a) The vulnerable lines involved. -- (b) Why it is present. -- (c) How an adversary could exploit it. -- (d) The simplest procedure to remove this vulnerability *while preserving the intended functionalities of the page*. If the vulnerability is not present, explain why.

Both in Snippet 2 and 3 b),c),d) see slides

[10 points] 35. Write an exploit to retrieve the password of the user Mathilda. Specify clearly all the steps and assumptions that are necessary to perform the attack correctly, including if the exploit needs interaction with the victim.

Use the SQL injection:
https://have.a.sitar.com/best_seller?month=' union (select password from Users where username='Mathilda'); --
The attack works by [...]

[10 points] 36. You really love seeing your friend Emily play. For this reason, you would like her to buy a bike. She already has enough money to buy one. Write an exploit to make Emily buy and receive a bike. Specify clearly all the steps and assumptions that are necessary to perform the attack correctly, including if the exploit needs interaction with the victim.

Use CSRF in Snippet 2
Send to your friend Emily the link https://have.a.sitar.com/buy?obj=Bike
The attack requires interaction and we assume Emily will click the link. The attack works by [...]

[20 points] 37. The developers of Have a Sitar have noticed that something weird is going on with the website. For this reason, they decide to take down the best_seller endpoint by modifying the first snippet and setting *page="Currently not available"*. (a) Is the mitigation effective against the exploits developed at point 35 and 36? (b) Is the mitigation effective in general against the class of vulnerability to which the exploits belongs? If there is a way to perform the exploit and bypass the mitigation, state the steps and assumptions clearly, including if the exploit requires interaction with the victim. If not, explain why.

The mitigation is not effective. They are mitigating the SQL injection exploit.
The exploit of question 35 won't work anymore as no output is available.
The mitigation is not effective.
Indeed, it is possible to perform a blind SQL injectionSee slides.

[20 points] 38. Assuming that you can create a maximum of two users and that you have no means to interact with other users from this website. Is there an exploit that you can think of that allows to gain free

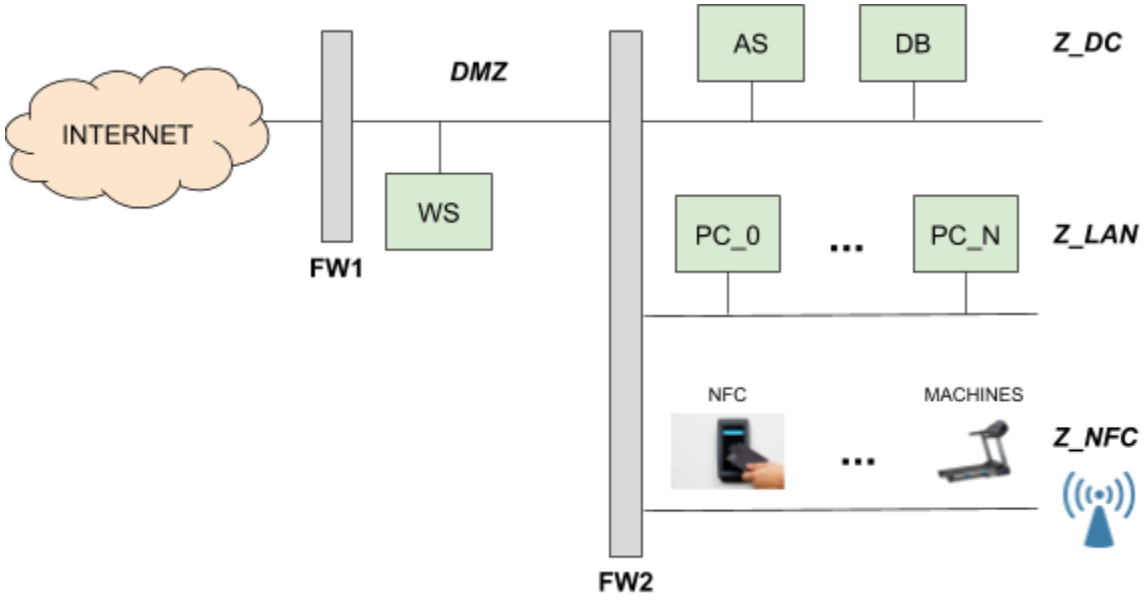
credits? If this is the case, write the exploit and specify all the steps and assumptions that are necessary to perform the attack correctly. If not, explain why.

Yes. There is a race condition in Snippet 3. In fact you can create two users that start with 5 credits each and start making concurrent donations with one user to the other. Let's say user 1 donates to user 2. If the race condition is triggered then user 1 has 0 credits and user 2 has 15 credits. If the race condition fails, user 1 has 0 credits and user 2 has 10 credits. The amount of money is either preserved or increased, you can always continue triggering the race and raise credits for free.

Exercise 3 (130 points)

"iFit" is a new and technologic gym in the center of the cosmopolitan city of NowhereLand. Its peculiarity is that customers can interact with every service without the need of any human staff whatsoever. The entrance door is equipped with a **NFC emitter** (a device similar to the ones installed on vending machines to use smart "keys" to pay), so clients can, at all times, enter the gym scanning a NFC "tag" (i.e., a badge) with their identifying information stored in it. The emitter, connected to the gym's network, reads the NFC tag and sends the client's data to an **application server** located on premise, inside the gym's datacenter. The application server, in turn, **queries a database** and returns a positive or negative message to the door depending on whether the client is allowed to enter.
The NFC technology used by "iFit" **encrypts the payload** saved in the tag (user ID, name, surname) with a **32-bit symmetric key**. NFC tags do not have any computing capability and are purely passive devices.
Inside the gym, each workout machine is connected to the Wi-Fi network, so that each user can use their personal badge (NFC tag) to authenticate: this way, the machine will **save data collected during the exercise session on the database** (by invoking an appropriate functionality of the **application server**). Note that the machines **cannot be used if there's no NFC tag** connected to them at all time during the exercise.
Once back home, the client can access the **website** of "iFit" (again, served by an on-premise web server) and authenticate themselves through their credentials to retrieve his/her results. From the website, clients can also pay and renew their subscriptions to the gym. For both operations, the web server invokes functionalities on the application server which, in turn, reads and stores data on the database server.
Finally, the office employees of iFit work from a **set of desktop computers** in the same local network. They need to be able to access the **application server**, the **NFC emitter** and the **gym machines**, as well as **browsing the Internet** (assume HTTP and HTTPS only).

The Figure below shows the current network layout of the iFIT network, including the (logical) locations of the firewalls and the network zones that have been implemented.



39 [10 points] Write the firewall rules, assuming firewalls to be stateful packet filters.

Firewall	Src IP	Src PORT	Direction of the 1st packet	Dst IP	Dst PORT	Policy	Description
FW1 (example)	*10.0.0.1 (example)*	*ANY*	*zone 1 -> zone 2*	*192.168.0.2 (example)*	*443*	*DENY*	*(example: the X server in zone 1 cannot contact the Y server)*
FW1	any	any	any	any	any	DENY	Default deny on FW1
FW2	any	any	any	any	any	DENY	Default deny on FW2
FW1	ANY	ANY	Internet -> DMZ	WS_IP	80, 443	ALLOW	WS exposes public facing website
FW2	WS_IP	ANY	DMZ -> Z_DC	AS_IP	AS_PORT	ALLOW	WS invokes the AS
FW2	ANY	ANY	Z_LAN -> Z_NFC	ANY	NFC_PORT	ALLOW	PCs can access the NFC devices
FW2	ANY	ANY	Z_LAN -> Z_DC	AS_IP	AS_PORT	ALLOW	PCs must access the AS
FW2	ANY	ANY	Z_LAN -> DMZ	ANY	80, 443	ALLOW	Internet
FW1	IPs in Z_LAN	ANY	DMZ->Internet	ANY	80, 443	ALLOW	Internet
FW2	ANY	ANY	Z_NFC -> Z_DC	AS_IP	AS_PORT	ALLOW	NFC devices invoke the AS

40 [10 points] Due to the outbreak of a worldwide pandemic, office employees are advised to work from home for the foreseeable future. For this reason, iFit has issued every employee with a corporate laptop, with the gym's management software pre-installed on it. The gym's managed software should access the application server in the gym's datacenter. Please describe (and motivate) any technology you should introduce and any network or firewall rule change that you should implement to enable this scenario.

Technology: VPN (a split tunnel configuration is enough). Changes to the network layout and firewall rules left as an exercise to the reader...

41 [10 points] Apparently a set of customers seems to be using **two different workout machines** at the same time. iFit is afraid that they're **sharing their credentials** with someone else. Considering that to use a machine a customer is required to leave his NFC tag on the machine itself and that the NFC tag's payload is encrypted, iFit is wondering how it is possible. How do you think these customers are scamming iFit? Would you be able to suggest a solution to avoid the problem?

Given the payload is simply encrypted with a symmetric key, it is possible that somebody duplicated the NFC tag by duplicating the encrypted payload. Alternatively, given the key is symmetric, the key is embedded in every reader thus it is possible that someone tampered with a reader to extract the key. Solution: the NFC tag should not be a purely passive device that transmits static data but should use a cryptographic protocol to authenticate the reader\emitter somehow. As per the symmetric key, it is possible to use asymmetric crypto to avoid having the static key hardcoded in each reader\emitter.

42. [10 points] Whoooops! iFit called you as a security analyst due to the fact that **their network has probably been compromised**. Their gym machines stopped communicating with their application server (IP address 18.194.76.151) after a network reset. They managed to get the last messages sent on the local network by the machines before they stopped responding (the following extract is taking in consideration one machine only):

1	aa:02:33:b1:ce:33 → ff:ff:ff:ff:ff:ff	**ARP**	*Who has 192.168.0.1? Tell 192.168.0.3*
2	03:cc:12:ff:d1:01 → aa:02:33:b1:ce:33	**ARP**	*192.168.0.1 is at bb:bb:bb:bb:bb:bb*
3	12:c9:87:e2:0d:ce → aa:02:33:b1:ce:33	**ARP**	*192.168.0.1 is at 12:c9:87:e2:0d:ce*
4	192.168.0.3 (dc:a9:04:7a:ce:29) → 18.194.76.151 (*bb:bb:bb:bb:bb:bb*)	**TCP**	*SYN*
5	192.168.0.3 (dc:a9:04:7a:ce:29) → 18.194.76.151 (*bb:bb:bb:bb:bb:bb*)	**TCP**	*SYN*
6	192.168.0.3 (dc:a9:04:7a:ce:29) → 18.194.76.151 (*bb:bb:bb:bb:bb:bb*)	**TCP**	*SYN*

You also receive the list of the MAC addresses of each device in the subnetwork:

- aa:02:33:b1:ce:00 to aa:02:33:b1:ce:40 - all the gym machines
- 03:cc:12:ff:d1:01 - Door computer
- 12:c9:87:e2:0d:ce - Network gateway

Identify which class of attack is going on in the network, and state how it does work in this particular scenario and what is the attack's intended purpose.

ARP poisoning, goal: denial of service

43. [10 points] Are you able to identify what is the attacker's machine (IP and\or MAC address)?
If so, describe how you are able to do so and state the attacker's IP and\or MAC address. If not, please explain why.

IP: no, attack is on layer 2
MAC: attacker's mac is 03:cc:12:ff:d1:01 hence the door computer but it can be spoofed easily. Thus, we cannot conclude anything.

44. [10 points] Now focus on *detecting when this attack is taking place*. Please outline how an intrusion detection system (IDS) can **detect** this attack class.
Assume the IDS is connected to a SPAN port of the local network switch (i.e., besides being able to send and receive packets on the local network, it also **receives a real-time log of the traffic passing through all the ports of the switch**).
If you think detection is not possible in this scenario, please explain why. Finally, motivate whether this same IDS setup could be tweaked so that it is able to **prevent or block** this attack.

Detection: yes, it is possible by observing ARP responses that are not coherent (e.g., multiple responses for the same request, gratuitous arp response with a "new" MAC address for an already seen IP)

Prevention: no. For example, by keeping a MAC\IP table and observing when there is a change in the MAC\ip association from a new arp response, it is possible to detect an ARP spoofing attempt. However, in this case there isn't much that can be done, except raising an alert and\or sending gratuitous ARPs to "override" the maliciously set entry, but this doesn't actually protect or block the malicious ARP response. It is possible to do some enforcement\filtering of ARP responses at the switch (non at the IDS) level.

[10 points] 45. Scared about the threat of ransomware blocking the gym's operation, iFit wants to implement a centralized **antimalware** and **data loss prevention** solution, without installing software on the employees' machines. This solution basically should analyze the content of **each HTTP response** for *known* malware or for strings matching personal data (e.g., credit card numbers). Please describe, **from a network infrastructure point of view**, all the technologies, network and firewall rule changes you need to implement in order to enable this scenario.

Install a web proxy with traffic interception capabilities (added, e.g., in the DMZ). Pay attention in the firewall rules to block all outgoing traffic towards the Internet not flowing through the proxy.

[5 points] 46. How does the previous answer change in case iFit wants to perform antimalware and data loss prevention checks on data exchanged over **HTTPS (TLS)**?

Need to perform MITM and deploy a trusted certificate in every computer. Not a problem as devices are company managed. Possibly problematic and\or unfeasible in case of personal devices.

[10 points] 47. Does the previous solution work without modifications for the "work from home" scenario? Why? If not, please describe a way to implement this solution also when employees are working from home, clearly describing all the assumptions and requirements.

No, it doesn't work, at home traffic doesn't flow through the corporate gateway. It is possible to implement a full tunnel VPN but users may be able (and\or required - e.g., to bypass a captive portal) to disconnect the VPN and browse the internet without it. At this point, sysadmins can lock down the PC configuring the VPN to auto-connect on startup and\or locking the browser down so that it can't bypass the corp proxy.

[20 points] 48. In the previous question, the antimalware solution was aimed at detecting known malware. Consider now the threat of polymorphic malware. Please describe why the previous solution is not effective against polymorphic malware, and propose a technique that the detection system can use in order to detect polymorphic malware. Finally, discuss whether this improved implementation of the antimalware solution is effective against metamorphic malware.

Polymorphic) Static (if possible) or dynamic unpacking (run the potential malware in a sandbox until it decrypts the payload and scan the decrypted payload in memory using signature based detection)
Metamorphic) Same technique not effective with metamorphic malware => behavioral analysis, e.g., detect sequence of potentially malicious syscalls, …

49. Do you want your exam to be corrected ? Select No, if you want to withdraw from the exam. If you click yes and your final grade is below 120 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).

- ❑ YES
- ❑ NO